

RESTFUL API

**OPENAPI, VALIDATION, ERROR CONTRACT VÅ
LOGGING**

AGENDA

BLOCK 1, 120 PHÚT

01

RESTful API và OpenAPI

02

Validation, status và error contract

03

Logging với request_id

04

Đề bài Cart API, bàn giao và nghiệm thu

WHAT: OPENAPI

Là gì: file YAML hoặc JSON mô tả path, method, parameter, body, response và lỗi.

01

Tài liệu: Swagger UI, Redoc hoặc Scalar sinh trang `/docs` từ file này.

02

Kiểm tra: validator dùng cùng file để chặn request sai và đối chiếu response.

03

Khác tài liệu viết tay: công cụ phát hiện được khi code lệch spec.

04

VÍ DỤ: MỘT OPERATION TRONG SPEC

```
/carts/{cartId}/items:  
  post:  
    operationId: addCartItem  
    parameters:  
      - $ref: '#/components/parameters/CartId'  
    requestBody:  
      required: true  
      content:  
        application/json:  
          schema: { $ref: '#/components/schemas/AddItemRequest' }  
    responses:  
      '201': { $ref: '#/components/responses/Cart' }  
      '400': { $ref: '#/components/responses/Error' }
```

VÍ DỤ: SCHEMA CỦA REQUEST BODY

```
components:
  schemas:
    AddItemRequest:
      type: object
      additionalProperties: false
      required: [product_id, quantity]
      properties:
        product_id: { type: string, format: uuid }
        quantity: { type: integer, minimum: 1, maximum: 10 }
```

- **Validator đọc schema này:** `quantity: 0`, `quantity: "2"` hoặc field lạ đều bị trả 400.

CONTRACT-FIRST VÀ CODE-FIRST

Contract-first

- **Thứ tự:** viết `openapi.yaml` trước, code sau.
- **Validation:** middleware đọc spec, ví dụ `express-openapi-validator`.

Code-first

- **Thứ tự:** viết schema trong code, sinh spec từ code.
- **Ví dụ:** `zod` kết hợp `@asteasolutions/zod-to-openapi`.

WHAT: VALIDATION Ở BOUNDARY

Schema kiểm tra được

- **Kiểu và định dạng:** `quantity` là integer, `cartId` là uuid.
- **Giới hạn:** `quantity` từ 1 đến 10, không có field lạ.

Cần code nghiệp vụ

- **Dữ liệu trong DB:** product có tồn tại và đang bán.
- **Trạng thái:** đủ tồn kho, cart chưa checkout.

REQUIRED, OPTIONAL VÀ NULL

Key có mặt hay không

- **Required:** key phải có trong object.
- **Optional:** key được phép vắng mặt.

Giá trị của key

- **Null:** chỉ hợp lệ khi schema cho phép `type: [string, 'null']`.
- **Client:** xử lý key vắng mặt khác với key bằng null.

WHAT: STATUS NÀO CHO LỖI NÀO

STATUS	KHI NÀO	VÍ DỤ TRONG BÀI
400	Request sai schema	<code>quantity</code> bằng 0
404	Resource trên URL không có trong DB	<code>cartId</code> không tồn tại
409	Xung đột với trạng thái hiện tại	Hết tồn kho
422	Body đúng schema, tham chiếu sai	Product ngừng bán

WHAT: ERROR CONTRACT

```
{  
  "code": "VALIDATION_ERROR",  
  "message": "Request body không hợp lệ",  
  "details": [{ "field": "quantity", "issue": "must be ≤ 10" }],  
  "request_id": "7f3c2a9e-1b4d-4c8a-9e21-5d6f0a3b8c10"  
}
```

- **code**: chuỗi cố định để client rẽ nhánh; `message` chỉ để hiển thị.
- **500**: cùng dạng JSON, không chứa stack trace, câu SQL hay secret.

WHAT VÀ WHY: LOGGING

Why: response lỗi chỉ cho client biết lỗi gì; log cho biết lỗi xảy ra ở đâu.

01

request_id: sinh cho mỗi request, trả trong header `X-Request-Id` và error body.

02

Gợi ý thư viện: `pino-http`, `winston` hoặc `morgan`; nhóm tự chọn.

03

Không ghi: password, token, header `Authorization`.

04

BÀI TẬP: CART API

NGƯỜI DÙNG: xem danh sách sản phẩm, tạo giỏ hàng, thêm, đổi số lượng và xóa sản phẩm trong giỏ.

01

NHÓM TỰ DỰNG: project RESTful API, DB (eg: PostgreSQL), migration, seed và script reset DB; thư viện do nhóm chọn.

02

CHƯA LÀM TUẦN NÀY: đăng nhập, checkout, thanh toán; cart được xác định bằng `cartId`.

03

DB: BẢNG PRODUCTS

```
CREATE TABLE products (  
  id          uuid PRIMARY KEY,  
  sku         text UNIQUE NOT NULL,  
  name        text NOT NULL,  
  price_cents integer NOT NULL CHECK (price_cents > 0),  
  stock       integer NOT NULL CHECK (stock ≥ 0),  
  is_active   boolean NOT NULL DEFAULT true  
);
```

- **Seed:** 5 product; 3 đang bán còn hàng, 1 có `stock = 0`, 1 có `is_active = false`.

DB: BẢNG CARTS VÀ CART_ITEMS

```
CREATE TABLE carts (  
  id      uuid PRIMARY KEY,  
  status text NOT NULL DEFAULT 'open'  
          CHECK (status IN ('open', 'checked_out'))  
);  
CREATE TABLE cart_items (  
  cart_id      uuid REFERENCES carts(id) ON DELETE CASCADE,  
  product_id   uuid REFERENCES products(id),  
  quantity     integer NOT NULL CHECK (quantity BETWEEN 1 AND 10),  
  PRIMARY KEY (cart_id, product_id)  
);
```

- **Seed:** 1 cart có `status = 'checked_out'` để thử lỗi ghi vào cart đã đóng.

ĐỀ BÀI: ENDPOINT PRODUCT VÀ CART

ENDPOINT	HÀNH VI
GET /products? limit=&offset=	Product đang bán; <code>limit</code> 1-50, mặc định 20
POST /carts	Tạo cart rỗng; 201, header <code>Location</code>
GET /carts/{cartId}	Cart, items và <code>subtotal_cents</code>

ĐỀ BÀI: ENDPOINT ITEM TRONG CART

ENDPOINT	BODY	THÀNH CÔNG
POST /carts/{cartId}/items	{ product_id, quantity }	201, trả cart
PATCH /carts/{cartId}/items/{productId}	{ quantity }	200, trả cart
DELETE /carts/{cartId}/items/{productId}	Không có	204

ĐỀ BÀI: RULE NGHIỆP VỤ VÀ MÃ LỖI

RULE	VI PHẠM THÌ TRẢ
Product tồn tại và đang bán	422 <code>PRODUCT_UNAVAILABLE</code>
Số lượng không vượt tồn kho	409 <code>INSUFFICIENT_STOCK</code>
Product chưa có trong cart khi thêm	409 <code>ITEM_ALREADY_IN_CART</code>
Chỉ ghi vào cart đang mở	409 <code>CART_CLOSED</code>

– **Lỗi schema và URL:** 400 `VALIDATION_ERROR` ; 404 `CART_NOT_FOUND` , `ITEM_NOT_FOUND` .

YÊU CẦU BÀN GIAO

HẠNG MỤC	YÊU CẦU
Setup	README ghi lệnh tạo DB, chạy migration, seed, reset và server
API	6 endpoint đúng method, path, status của đề bài
Documentation	OpenAPI 3.1 đủ mọi status; <code>/docs</code> chạy được; có example
Validation	Chạy từ cùng nguồn schema với spec; request sai không chạm DB
Error và logging	Mọi lỗi theo error contract, kể cả 500; gợi ý log kèm <code>request_id</code>
Evidence	Script chạy lại được cho mọi dòng của ma trận nghiệm thu

MA TRẬN NGHIỆM THU: REQUEST SAI

KỊCH BẢN	KẾT QUẢ CẦN THẤY
<code>quantity</code> bằng 0, 11 hoặc <code>"2"</code>	400 <code>VALIDATION_ERROR</code> , <code>details</code> chỉ <code>quantity</code>
Thiếu <code>product_id</code> ; body có field lạ	400 <code>VALIDATION_ERROR</code> ; DB không đổi
<code>cartId</code> không phải uuid	400 <code>VALIDATION_ERROR</code>
<code>cartId</code> hợp lệ nhưng không tồn tại	404 <code>CART_NOT_FOUND</code>

MA TRẬN NGHIỆM THU: NGHIỆP VỤ VÀ SERVER

KỊCH BẢN	KẾT QUẢ CẦN THẤY
Thêm 2 product rồi GET cart	subtotal_cents bằng tổng tính tay từ seed
Product ngừng bán; vượt stock	422 PRODUCT_UNAVAILABLE ; 409 INSUFFICIENT_STOCK
Ghi vào cart đã checkout	409 CART_CLOSED
Tắt PostgreSQL rồi gọi API	500 đúng error contract, không có stack trace
So response thật với spec	Mọi response ở hai bảng khớp schema

CÁC LỖI THƯỜNG GẶP

Hai nguồn schema: spec viết một kiểu, validation trong code viết kiểu khác.

01

Lỗi thư viện lọt ra: validator trả format riêng, không theo error contract.

02

Tin client: nhận `price` từ body thay vì đọc từ DB.

03

Dựa vào lỗi DB: vi phạm `CHECK` constraint thành 500 thay cho 400.

04

PRESENTATION

PROBLEM: Cart API cần gì để nhóm khác dùng được mà không đọc source.

01

SOLUTION: contract-first hay code-first, thư viện đã chọn và lý do.

02

DEMO: gửi một request sai từ `/docs`, chỉ response lỗi và dòng log có cùng `request_id`.

03

EVIDENCE VÀ TRADE-OFF: kết quả ma trận; kịch bản chưa kiểm tra.

04

Q & A